IE 678: Deep Learning

Assignment No. 1 (FSS 2026)

Prof. Dr.-Ing. Margret Keuper

University of Mannheim
Chair of Machine Learning

Task	Maximum	Points
1	6	.
2	18	.
3	8	.
Total	32	.

Rules for Submission

a) Hand-in your solution via ILIAS until the date specified there. This is a hard deadline.

b) Pair submission:

   (i) We strongly recommend to work on this assignment in pairs—i.e, with one (and only one) additional student—and then hand in a pair submission.

   (ii) Both students must submit exactly the same contents.

c) Code:

   (i) You must submit all relevant code files (Python scripts etc.) that we require to run your code and to replicate your results.

   (ii) In particular: You must submit all Python scripts that we provided to you.

   (iii) You must include a PDF of every Jupyter notebook in which the most relevant cells are executed (i.e., make sure to disable excessively verbose cells that inflate the page count).

   (iv) It suffices to fill out the "holes" that are marked in the code fragments.

d) Non-code parts of assignment:

   (i) You must provide your answers by filling out the provided solution LaTeX file.

   (ii) You must submit a single PDF file (do not submit LaTeX source code). All answers must be provided *inside* the boxes (i.e., no resizing of boxes, no overflow, no additional pages, no modification of font (size) etc.) specified therein.

   (iii) You may add any LaTeX packages that you absolutely need.

   (iv) You must fill out the AI declaration at the end of the solution file following the provided instructions.

   (v) The solution PDF and all notebooks must clearly report the name and matriculation number of both students right at the beginning.

   (vi) Name your solution PDF as follows:
   `a0[assignment number]-[ilias name 1]-[ilias name 2].pdf`
   For instance: `a01-gmayer-tswift.pdf`

We reserve the right to refuse to grade any assignment that violates any of these rules.

Task 1: Implement an MLP — 6 Points

In this assignment, we explore simple feedforward networks in PyTorch. To get used to PyTorch, you **must not use NumPy** in your solutions to this assignment. Instead, you **must use PyTorch** (and plain Python, of course) throughout.

a) Examine the class `LogisticRegression` provided to you. Pay attention to (i) how the parameters are stored, (ii) how the calculations in the forward pass are performed, and (iii) how the model is initialized and used.

b) Complete the implementation of the class `MLP` provided to you. The expected functionality is documented directly in the Python file.

Requirements. You must implement the MLP without using pre-canned PyTorch neural network functionality. In particular, you must not use any layer from `torch.nn`.[1] You can (and should) use `torch.nn.Parameter` and `torch.nn.Module`, however. (4 points)

c) Expand the implementation of the class `MLP` such that:

(i) When a vector ($\boldsymbol{x}$) is provided, it computes the network's output ($\boldsymbol{y} = f(\boldsymbol{x})$) as a vector.

(ii) When a matrix is provided, it computes the network's output for each row as a matrix, i.e.,

$$\boldsymbol{X} = \begin{pmatrix} \boldsymbol{x}_1^\top \\ \vdots \\ \boldsymbol{x}_N^\top \end{pmatrix} \to \begin{pmatrix} f(\boldsymbol{x}_1)^\top \\ \vdots \\ f(\boldsymbol{x}_N)^\top \end{pmatrix} = \boldsymbol{Y}.$$

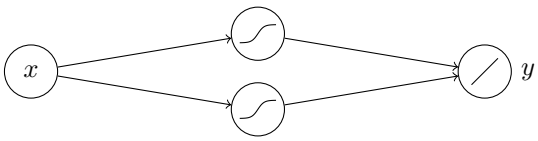
Models are commonly implemented like this in PyTorch: the first dimension ("batch dimension") corresponds to an example, the remaining dimensions then hold the actual values of that example.

Requirements. Do not use `torch.vmap` in your solution, but directly implement the required operations efficiently. (2 points)

[1] PyTorch provides pre-canned layer implementations in classes such as `torch.nn.Linear` (see `https://pytorch.org/docs/stable/nn.html`the documentation for an overview). We'll cover those later in the course; here we stick to the basics.

Task 2: Experiment with MLPs — 18 Points

In this task, we look at regression using MLPs of form



where we vary the number of units in the hidden layer. Luckily, you have implemented such networks in the previous tasks already.

We use a 1-dimensional dataset $\mathcal{D}_1$ (`X1` and `y1`, as well as `X1test` and `y1test`).

Hint. We provide the function `saveplot` that allows you to export a plot from one of your notebooks. You may use, e.g., `\includegraphics[width=200pt]{a01_2_data.pdf}`, to include the figure `figures/a01_2_data.pdf` inside an answer box.

a) Look at the training data and **conjecture** how a fit for an FNN with (i) zero, (ii) one, (iii) two, and (iv) three hidden neurons would look like *before* you proceed with this task.

   Hint. Even a wrong answer may be a good conjecture, i.e., receive points. (4 points)

b) The Python file provides code for training based on Scipy's BFGS optimizer. Train an FNN with two hidden neurons, determine the mean squared error (MSE) on the training and the test data, and plot.

   (i) **Discuss** whether the result is as you expected (cf. your answer to a)).

   (ii) Now repeat training multiple times. **Describe** what happens. **Explain** why this happens. (4 points)

c) Train a FNN with 1, 2, 3, 10, 50, and 100 hidden neurons. In each case, determine the MSE on the training and the test dataset. Then plot the dataset as well as the predictions of each FNN on the test set into a single plot.

   (i) **Describe** what happens when the number of hidden neurons increases.

   (ii) **Discuss** whether this is this what you expected (cf. your answer to a)). (4 points)

d) Train a FNN with 2 hidden neurons and visualize the output of the hidden neurons (=distributed representation / embedding, code provided). Then visualize the output of the hidden neurons scaled by the weight of their respective connection to the output (code also provided). As before, you may want to repeat this process multiple times.

   (i) **Explain** how the FNN obtains its flexibility.

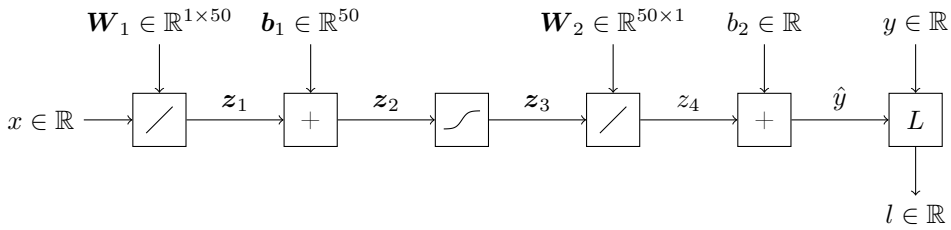
   (ii) **Discuss** whether the distributed representation is intuitive.

   (iii) Now repeat (i) and (ii) with 3 hidden neurons, then 10 hidden neurons. **Discuss** these cases in comparison to your answers to (i) and (ii). (6 points)

e) (Optional.) Repeat the above experiments with different optimizers; e.g., try the Adam optimizer provided with PyTorch (code provided). Also experiment with different numbers of hidden layers and/or with different types of units in the hidden layers. For example, it is instructive to repeat subtasks a)—d) with ReLU units. **Describe** and **discuss** your findings in comparison to your findings in a)–d). (0 points)

Task 3: Backpropagation 8 Points

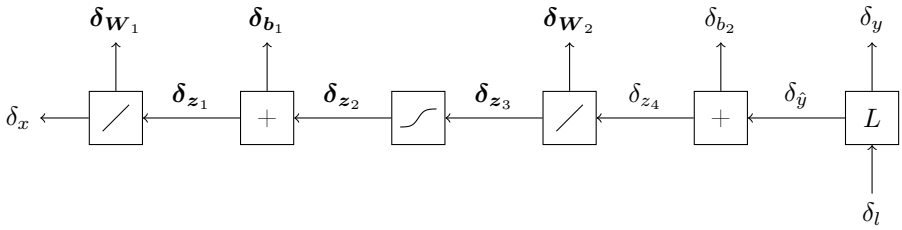
Consider the network of the previous task with a single hidden layer of size $Z = 50$. In this task, we are going to compute various gradients of the network output with respect to its inputs and weights manually. The network is represented by the following compute graph:



The $\boxed{/}$ operations perform matrix-vector multiplications (e.g., $\boldsymbol{z}_1 = \boldsymbol{W}_1^\top x$), the $\boxed{+}$ operations element-wise addition (e.g., $\boldsymbol{z}_2 = \boldsymbol{z}_1 + \boldsymbol{b}_1$), and the $\boxed{\sigma}$ operations applies the logistic function element-wise (e.g., $[\boldsymbol{z}_3]_k = \sigma([\boldsymbol{z}_2]_k)$). Unit $\boxed{L}$ computes the squared loss between prediction $\hat{y}$ and target y:

$$L(y, \hat{y}) = (y - \hat{y})^2.$$

The backward graph for this network looks as follows:



We provide code to extract the model parameters from a trained model, run the model using PyTorch on a single input `x`, and extract (some of) the gradients using PyTorch's autograd.

a) Compute the results of the forward pass directly using basic PyTorch operations. Your code must only access model inputs, targets, and parameters (i.e., `x`, `y`, `W1`, `b1`, `W2`, `b2`). **Give** derivations and resulting formulas (not Python code) for each of these quantities. (3 points)

b) Compute the results of the backward pass directly using basic PyTorch operations (i.e. without using autograd). Your code must only access model inputs, targets, parameters, and forward pass outputs (i.e., `x`, `y`, `W1`, `b1`, `W2`, `b2`, `z1`, `z2`, `z3`, `z4`, `yhat`). **Give** derivations and resulting formulas (not Python code) for each of these quantities. (5 points)